

第 9 章：TRPO (Trust Region Policy Optimization, 信赖域策略优化)、PPO (Proximal Policy Optimization, 近端策略优化) 与策略更新约束

9.1 本章符号说明

符号/缩写	English	中文	含义
TRPO	Trust Region Policy Optimization	信赖域策略优化	用 KL 约束限制策略更新幅度。
PPO	Proximal Policy Optimization	近端策略优化	用 clipping objective 近似限制策略更新幅度。
KL	Kullback-Leibler Divergence	KL 散度	衡量新旧策略分布差异。
θ / θ_{old}	New / Old Policy Parameters	新 / 旧策略参数	当前要优化的策略参数和采样数据时的旧参数。
$\rho_t(\theta)$	Probability Ratio	概率比	新策略和旧策略对同一动作的概率比；使用 ρ 避免与即时 reward R_{t+1} 混淆。
$A_t / \hat{A}_t$	Advantage Estimate	优势估计	判断当前动作应提高还是降低概率的样本信号；两种写法都表示估计量，不是真实 A_{π^*} 。
$L^{CLIP}(\theta)$	Clipped Objective	截断目标函数	PPO 用来限制策略更新的 surrogate objective。
ϵ_{clip}	Clipping Range	截断范围	PPO 中控制 ratio 允许变化范围的超参数；不是 epsilon-greedy 的探索率。
η	Policy Learning Rate	策略学习率	控制一次策略参数更新的步长。
δ_{KL}	Trust-region Radius	信赖域半径	TRPO 对平均 KL divergence 设置的上界。
L_{policy} / L_{value}	Policy Loss / Value Loss	策略损失 / 价值损失	PPO 总 loss 的两个主要部分。
$V(s)$	Critic Value Estimate	Critic 价值估计	PPO 的 value head 对状态 s 的价值预测。
V_t^{target}	Value Target	价值目标	用 return 或 bootstrap 构造、用于训练 critic 的监督目标。上标 target 用来区别当前预测 $V(s_t)$ 。
c_v / c_e	Value / Entropy Coefficient	价值 / 熵系数	分别控制 value loss 和 entropy bonus 在总 loss 中的权重。
$H(\pi)$	Entropy	熵	鼓励策略保持探索的正则项。
behavior policy	Behavior Policy	行为策略	产生当前 rollout 数据的旧策略。

符号/缩写	English	中文	含义
target policy	Target Policy	目标策略	当前正在优化的新策略。
on-policy	On-policy Learning	同策略学习	主要用当前策略或近似当前策略采样的数据更新。
off-policy	Off-policy Learning	异策略学习	用其他策略产生的数据学习目标策略。
CLIP	Clipped Objective Label	截断目标标记	L^{CLIP} 中的上标, 表示 PPO 的 clipped surrogate objective。
GAE	Generalized Advantage Estimation	广义优势估计	PPO 中常用的 advantage 估计方法。
DQN	Deep Q-Network	深度 Q 网络	PPO 对比中的 value-based deep RL 算法。
RL / RLHF	Reinforcement Learning / Reinforcement Learning from Human Feedback	强化学习 / 基于人类反馈的强化学习	PPO 既用于 RL, 也常出现在 RLHF 流程中。

9.2 本章目标

本章学习现代 on-policy deep RL 中最常用的一类算法。你需要掌握：

- 为什么 policy gradient 需要限制更新步长。
- importance sampling ratio。
- TRPO 的 trust region 思想。
- PPO clipping objective。
- PPO 的训练细节。
- PPO 的优缺点。

9.3 本章主线

Actor-Critic 解决了 policy gradient 高方差问题，但策略更新仍可能一步走太大，导致新策略和采样数据对应的旧策略差异过大。本章专门处理“更新幅度控制”：TRPO 用 trust region 和 KL 约束，PPO 用 clipping objective 做更易实现的近似。

9.4 本章新增概念说明

本章核心是“限制策略更新幅度”，先把 PPO/TRPO 相关名词定义清楚：

名词	中文	先记住什么
old policy	旧策略	采样 rollout 时使用的策略。
new policy	新策略	当前正在优化、准备替换旧策略的策略。
behavior policy	行为策略	在 PPO 里通常就是采样数据的 old policy。
target policy	目标策略	当前想优化得到的 new policy。
KL divergence	KL 散度	衡量新旧策略分布差异，越大说明策略变化越大。

名词	中文	先记住什么
trust region	信赖域	限制策略每次只能在可信范围内小步更新。
importance sampling ratio	重要性采样比率	新策略概率除以旧策略概率，用旧数据估计新策略目标。
surrogate objective	替代目标	用已有 rollout 近似优化真实策略目标的目标函数。
clipping	截断	限制 ratio 过大或过小，防止策略更新过猛。
entropy bonus	熵奖励	鼓励策略保持一定随机性，避免过早收敛。
advantage normalization	优势归一化	把 advantage 标准化，让训练尺度更稳定。

9.5 为什么要约束策略更新

9.5.1 为什么要限制策略更新

Policy Gradient 直接更新策略参数：

$$\theta \leftarrow \theta + \text{eta} \nabla J(\theta)$$

但策略优化很敏感。如果一次更新太大，新的 policy 和采样数据对应的旧 policy 差异过大，可能导致性能崩掉。

原因：

- on-policy 数据只对采样它的旧策略可靠。
- 策略变化太大时，旧数据不能准确估计新策略表现。
- neural policy 的参数小变化可能导致动作分布大变化。

因此 TRPO/PPO 都试图限制新旧策略距离。

9.5.2 旧策略数据怎样评估新策略：Ratio 从哪里来

PPO 的数据是旧策略 $\pi_{\theta_{old}}$ 采样出来的，但我们想评估新策略 π_{θ} 。如果完全重新采样，成本很高；如果直接把旧数据当新数据，又会有分布偏差。

importance sampling 的基本想法是：

$$\mathbb{E}_{a \sim \pi_{\theta}}[f(a)] = \mathbb{E}_{a \sim \pi_{old}}[\pi_{\theta}(a|s) / \pi_{old}(a|s) f(a)]$$

所以出现 ratio：

$$\rho_t(\theta) = \pi_{\theta}(a_t|s_t) / \{\pi_{\theta_{old}}(a_t|s_t)\}$$

它表示在同一个 (s_t, a_t) 上，新策略相对旧策略把动作概率改了多少：

- $\rho_t=1$ ：动作概率未变。
- $\rho_t>1$ ：新策略更偏好该动作。
- $\rho_t<1$ ：新策略降低了该动作概率。

使用第 8 章已经算好的 advantage estimate, surrogate objective 写成：

$$L^{surrogate}(\theta) = \mathbb{E}_t[\rho_t(\theta)\hat{A}_t]$$

- $\hat{A}_t > 0$: 希望增大 ρ_t , 提高好动作概率。
- $\hat{A}_t < 0$: 希望减小 ρ_t , 降低差动作概率。

这里要明确输入依赖: $\hat{A}_t$ 是旧策略 rollout 上计算出的固定学习信号, 通常来自 GAE; 优化 Actor 时对它 stop gradient。

9.5.3 为什么有 Ratio 仍然不够

Action probability ratio 修正了“同一状态下动作概率变了多少”, 但普通 surrogate objective 仍有近似:

- Rollout 中的状态由旧策略访问得到。
- 新策略变化后, 未来访问的状态分布也会变化。
- 单步 action ratio 没有完整修正长轨迹状态分布的变化。
- ρ_t 过大或过小时, 少数样本会主导梯度, 估计方差上升。

因此 ratio 不是允许无限复用旧数据的许可证。如果新旧策略差异太大, 旧 rollout 对新策略的估计仍然不可靠。

这就引出下一节:

普通 Policy Gradient: 沿梯度更新

- > ratio: 有限利用旧策略 rollout 评估新策略
- > ratio 变化太大: surrogate 不再可靠
- > TRPO: 显式限制 KL trust region
- > PPO: 用 clipping 做一阶近似

9.6 Trust Region 与 PPO 方法

9.6.1 TRPO

TRPO 是 Trust Region Policy Optimization。

它希望在提升 surrogate objective 的同时, 限制新旧策略的 KL divergence:

$$\begin{aligned} \max_{\theta} \quad & \mathbb{E}[\rho_t(\theta) A_t] \\ \text{subject to} \quad & \mathbb{E}[KL(\pi_{old}(\cdot|s_t) \parallel \pi_{\theta}(\cdot|s_t))] \leq \delta_{KL} \end{aligned}$$

核心思想:

每次策略更新不能离旧策略太远, 保证更新在 trust region 内。

优点:

- 理论上更稳定。
- 避免灾难性大步更新。

缺点:

- 实现复杂。
- 需要额外处理策略分布的曲率、更新方向和可行步长。
- 工程成本较高。

这三个词的关系是：

组件	中文含义	在 TRPO 中做什么
second-order approximation	二阶近似	用 KL 对参数的局部曲率描述“策略分布变了多远”，而不只看普通梯度方向。
conjugate gradient	共轭梯度法	近似求解由曲率矩阵决定的更新方向，避免显式构造和求逆巨大矩阵。
line search	线搜索	沿候选方向缩小步长，直到 surrogate 改善且 KL constraint 真正满足。

所以 TRPO 的稳定性不是只来自写了一个 KL 符号，而是需要一套求方向、定步长和验约束的优化过程。

9.6.2 PPO

PPO 是 Proximal Policy Optimization。它保留了限制策略更新幅度的思想，但实现更简单。

PPO clipped objective:

$$L^{CLIP}(\theta) = \mathbb{E}[\min(\rho_t(\theta) A_t, \text{clip}(\rho_t(\theta), 1 - \epsilon_{clip}, 1 + \epsilon_{clip}) A_t)]$$

其中：

$$\rho_t(\theta) = \pi_{\theta}(a_t | s_t) / \pi_{\theta_{old}}(a_t | s_t)$$

ϵ_{clip} 常用 0.1 或 0.2。

9.6.3 PPO Clipping 直觉

当 $A_t > 0$ ：

- 增大动作概率是好事。
- 但 ratio 超过 $1 + \epsilon_{clip}$ 后，收益被截断。
- 防止过度增加该动作概率。

当 $A_t < 0$ ：

- 降低动作概率是好事。
- 但 ratio 低于 $1 - \epsilon_{clip}$ 后，收益被截断。
- 防止过度降低该动作概率。

一句话：

PPO clipping 允许策略朝 advantage 指示的方向更新，但限制更新幅度，避免新旧策略差异过大。

9.6.4 Clipping 分情况推理

PPO 的 min 不是为了让公式复杂，而是为了“好方向可以走，但不要走太猛”。

9.6.4.1 情况 1: $A_t > 0$

动作比平均水平好，应该提高它的概率。未截断目标是：

$$\rho_t A_t$$

当 ρ_t 从 1 增大时，目标变大，表示鼓励新策略更常选这个动作。但如果 $\rho_t > 1 + \epsilon_{clip}$ ，说明概率已经提高太多。此时 clipped 项固定在：

$$(1 + \epsilon_{clip}) A_t$$

再提高概率也不会得到更多收益，梯度被压住。

9.6.4.2 情况 2: $A_t < 0$

动作比平均水平差，应该降低它的概率。因为 A_t 是负数，减小 ρ_t 会让 $\rho_t A_t$ 变大，看起来是好事。但如果 $\rho_t < 1 - \epsilon_{clip}$ ，说明概率已经降低太多。clipped 项固定在：

$$(1 - \epsilon_{clip}) A_t$$

PPO 取 min，会阻止继续通过过度降低概率来“刷目标”。

因此 clipped objective 同时限制两种过度更新：

- 好动作概率不能一下子加太多。
- 坏动作概率不能一下子降太多。

这就是 PPO 比普通 policy gradient 更稳的核心。

9.6.5 PPO Loss 组成

常见 PPO 总 loss：

$$L = L_{policy} + c_v L_{value} - c_e Entropy$$

如果按最小化写：

$$L_{policy} = - \mathbb{E}[\min(\rho_t A_t, \text{clip}(\rho_t, 1 - \epsilon_{clip}, 1 + \epsilon_{clip}) A_t)]$$

Value loss：

$$L_{value} = \mathbb{E}[(V(s_t) - V_t^{target})^2]$$

Entropy bonus：

$$Entropy = \mathbb{E}[H(\pi(\cdot | s_t))]$$

实际实现中还常见：

技巧	做法	解决的问题
advantage normalization	在 rollout batch 内把 advantage 减均值、除标准差	稳定 Actor loss 的数值尺度；不改变样本好坏排序。
value clipping	限制新旧 value prediction 的变化	防止 Critic 一次更新过猛；它与 policy ratio clipping 不是同一项。
gradient clipping	限制整体梯度范数	避免异常 minibatch 造成极大参数更新。

技巧	做法	解决的问题
target KL early stopping	实测新旧策略 KL 超阈值时提前结束当前 epoch	Clipping 不是严格 KL 约束，因此增加额外安全检查。
learning-rate annealing	训练后期逐步降低学习率	让后期策略更新更细，减少振荡。

9.6.6 PPO 训练流程

```

for each iteration:
    collect rollout using pi_old
    compute returns and advantages, often with GAE
    for several epochs:
        shuffle rollout data
        split into minibatches
        update policy with clipped objective
        update value function
    set pi_old <- current policy

```

注意：

PPO 是 on-policy，但它会对同一批 rollout 做多个 epoch 的 minibatch 更新。clipping 机制就是为了让这种有限复用不至于让策略偏离太远。

9.6.7 PPO 一轮训练的推理顺序

从实现角度，PPO 一轮迭代可以按下面理解：

1. 用当前策略采样 rollout，此时这份数据绑定的是 π_{old} 。
2. 用 reward 和 critic 计算 return。
3. 用 GAE 计算 advantage，得到每个动作“比平均水平好多少”。
4. 保存旧策略对已采样动作的 log probability。
5. 更新时重新计算新策略 log probability。
6. 用新旧 log probability 的差得到 ratio。
7. 用 clipped objective 更新 actor。
8. 用 return target 更新 critic。
9. 用 entropy bonus 防止策略过早塌缩。

这里最关键的是第 4 到第 7 步：PPO 不是只看当前策略输出，而是一直比较“新策略相对采样时旧策略改了多少”。

9.7 PPO 使用场景和对比

9.7.1 PPO 为什么常用

优点：

- 实现比 TRPO 简单。
- 稳定性好。
- 对超参数相对不那么敏感。
- 离散和连续动作都能用。
- 成为很多 RLHF 早期流程中的常见选择。

缺点：

- on-policy, sample efficiency 不如 off-policy 方法。
- 需要大量环境交互。
- 对 reward scaling、advantage normalization 仍然敏感。
- clipping 不是严格 trust region, 仍可能出现不稳定。

9.7.2 PPO vs DQN

维度	DQN	PPO
类型	value-based	policy-based / actor-critic
动作空间	离散	离散和连续
策略	由 Q argmax 导出	直接学习 policy
数据使用	off-policy	on-policy
样本效率	较高	较低
稳定性	需要 replay 和 target network	clipping + GAE 较稳定

9.8 本章例子：PPO clipping 怎么工作

假设某个动作在旧策略下概率是 0.20, 新策略下概率是 0.26。

importance ratio 是：

$$\rho_t = 0.26 / 0.20 = 1.3$$

如果 $\epsilon_{clip} = 0.2$, 允许范围是：

$$[0.8, 1.2]$$

9.8.1 Advantage 为正

如果 $A_t > 0$, 说明这个动作比预期好, 应该提高概率。但 ratio 已经到 1.3, 超过 1.2, PPO 会把收益截断在 1.2, 防止策略一步变太多。

9.8.2 Advantage 为负

如果 $A_t < 0$, 说明这个动作不好, 应该降低概率。但如果新策略把概率降得过猛, ratio 低于 0.8, 也会被 clip。

9.8.3 RLHF 例子

在 RLHF 里, 策略是语言模型, 动作可以理解成生成 token。PPO 更新时：

- reward model 给回答打分。
- critic 估计 value。
- advantage 判断这段输出比预期好还是差。
- KL 或 clipping 防止模型偏离参考模型太远。

面试表达：

PPO 的例子要讲清楚 ratio。它不是禁止策略更新, 而是限制同一批数据上新旧策略概率比变化过大, 避免 policy gradient 一步把策略推崩。

9.9 本章面试高频问题

9.9.1 PPO 解决了什么问题？

PPO 解决普通 policy gradient 更新过大导致策略崩掉的问题。它通过 clipping objective 限制新旧策略概率比的变化，使策略更新更稳定。

9.9.2 PPO 的 ratio 是什么？

$$\rho_t(\theta) = \pi_{\theta}(a_t | s_t) / \pi_{old}(a_t | s_t)$$

它衡量新策略相对于采样数据的旧策略，对同一动作概率改变了多少。

9.9.3 PPO clipping objective 的直觉？

如果 advantage 为正，允许提高动作概率，但超过 $1 + \epsilon_{clip}$ 后不再给额外收益。如果 advantage 为负，允许降低动作概率，但低于 $1 - \epsilon_{clip}$ 后不再给额外收益。

9.9.4 PPO 和 TRPO 的区别？

TRPO 通过 KL constraint 显式限制策略更新，理论更严谨但实现复杂。PPO 用 clipping 或 KL penalty 近似 trust region 思想，实现简单，工程上更常用。

9.9.5 PPO 是 on-policy 还是 off-policy？

PPO 通常是 on-policy。虽然它会对同一批 rollout 做多个 epoch 更新，但数据仍来自当前旧策略，且通过 ratio 和 clipping 控制偏移。

9.10 本章练习

1. 手写 PPO clipped objective。
2. 分别解释 $A_t > 0$ 和 $A_t < 0$ 时 clipping 的效果。
3. 对比 TRPO 和 PPO。
4. 说明 PPO 中 GAE、entropy bonus、advantage normalization 的作用。

▼ 参考答案（点击展开）

1. **PPO clipped objective**: $L^{CLIP}(\theta) = \mathbb{E}_t[\min(\rho_t(\theta)A_t, \text{clip}(\rho_t(\theta), 1-\epsilon_{clip}, 1+\epsilon_{clip})A_t)]$ ，其中 $\rho_t(\theta) = \pi_{\theta}(a_t | s_t) / \pi_{old}(a_t | s_t)$ 。实现成最小化 loss 时通常在该目标前加负号。
2. **Clipping 的两种情况**：当 $A_t > 0$ ，该动作比平均好，算法想提高其概率；若 ratio 超过 $1+\epsilon_{clip}$ ，收益被截住，避免提高过多。当 $A_t < 0$ ，算法想降低该动作概率；若 ratio 低于 $1-\epsilon_{clip}$ ，目标被截住，避免一次把概率压得过低。`min` 与 advantage 的符号共同形成这个效果。
3. **TRPO vs PPO**：TRPO 显式约束平均 KL divergence，并用二阶近似和共轭梯度求解，理论动机清晰但实现复杂。PPO 用 probability ratio clipping 或 KL penalty 近似限制更新，可用普通 minibatch SGD，工程实现更简单。二者通常都是 on-policy。
4. **三个组件的作用**：GAE 在 advantage 估计的 bias 和 variance 间折中；entropy bonus 防止 policy 过早变得确定、维持探索；advantage normalization 让一个 batch 内的更新尺度更稳定，降低学习率对 reward 量纲的敏感性，但它不改变 advantage 的相对排序。